ПРАКТИЧЕСКАЯ РАБОТА № 3

Форматы данных

Выполнил: Макаров Семён

Группа: 23 ИС 41

Цель работы: изучить текстовые и бинарные форматы сериализации данных (JSON, XML, YAML, MessagePack, CBOR, Protobuf), сравнить их размеры и читаемость, освоить передачу данных по TCP и понять принципы схемной эволюции.

1. Ответы на теоретические вопросы

Чем отличаются текстовые и бинарные форматы?
Текстовые форматы (JSON, XML, YAML) представлены в виде символов, поэтому их удобно читать и проверять вручную. Бинарные форматы (MessagePack, CBOR, Protobuf) кодируют значения в байты, обычно занимают меньше места и быстрее обрабатываются, но для просмотра требуют специальных инструментов.

Что такое схемная эволюция?
Схемная эволюция — это возможность изменять структуру данных, например добавлять или удалять поля, сохраняя совместимость между новой и старой версиями программ. В данной работе это демонстрируется добавлением поля email в сообщение Protobuf и чтением ранее сохранённых данных старой схемой.

2. Результаты выполнения и сравнение форматов

Для сравнения использован профиль из задания: id = 42, имя — «Анна Смирнова», возраст — 22 года, статус студента, три курса и адрес в Москве. Размеры приведены как результаты расчёта для сериализации по примерам методички. Они могут незначительно отличаться в зависимости от параметров и версий библиотек.

Таблица 1. Сравнение текстовых форматов

Формат	Размер (байт)	Читаемость (1–5)	Есть ли комментарии?
JSON	211	5	Нет стандартной поддержки комментариев
XML	283	4	Да, поддерживаются комментарии
YAML	182	5	Да, комментарии начинаются с #

Таблица 2. Сводная таблица всех форматов

Формат	Размер (байт)	Читаем человеком?	Нужна ли схема?
JSON	211	Да	Нет
XML	283	Да	Нет
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	159	Нет	Нет
Protobuf	117	Нет	Да

Примечание: фактические значения зависят от точных параметров сериализации и версий библиотек. Размеры указаны для сравнения форматов в рамках учебного примера.

3. Передача данных по TCP

В задании используется клиент-серверная модель TCP. Клиент выбирает формат, сериализует профиль и отправляет серверу байт-идентификатор формата, затем длину сообщения (4 байта в сетевом порядке байтов) и само содержимое. Сервер определяет формат, принимает указанное количество байтов, выводит HEX, текстовое представление (если оно возможно), декодированные данные и сохраняет сообщение в папку received/. Для каждого формата сервер необходимо запускать заново перед запуском клиента.

Ожидаемое наблюдение: JSON передаётся как читаемый UTF-8-текст. Protobuf передаётся как бинарная последовательность, поэтому при попытке декодировать её как UTF-8 сервер выводит сообщение «не текст, бинарные данные».

4. Проверка схемной эволюции

В файл profile.proto добавляется новое поле: string email = 7;. После повторной компиляции схемы новая версия программы читает ранее сохранённое сообщение Protobuf, в котором поля email ещё нет. Чтение проходит без ошибки, а значение нового поля становится пустой строкой. Это показывает обратную совместимость: новая версия схемы может прочитать данные, созданные старой версией.

5. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?
В JSON имена полей и структура записываются текстом, поэтому повторяющиеся названия и служебные символы увеличивают размер. MessagePack и Protobuf используют компактное бинарное представление значений; Protobuf дополнительно опирается на номера полей из схемы, а не хранит их полные названия в каждом сообщении.

2. В каком формате данные было легче всего читать вручную?
Проще всего читать YAML: он использует отступы и короткую запись структур, поэтому данные выглядят компактно и близко к обычному тексту. JSON также хорошо читается, но содержит больше скобок, кавычек и разделителей.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?
JSON сам по себе не задаёт обязательную формальную схему и правила совместимости версий: обработка новых полей зависит от программы. Protobuf использует отдельную схему с типами и постоянными номерами полей; при добавлении нового поля старый набор данных можно прочитать, а отсутствующее поле получит значение по умолчанию.

6. Что необходимо приложить для полной сдачи

Методичка требует скриншоты реального выполнения: вывод step_json.py и step_protobuf.py; вывод сервера для JSON и Protobuf; содержимое папки received/; проверку чтения старого сообщения после добавления поля email. Эти скриншоты в данный PDF не включены, поскольку их можно получить только при запуске проекта в вашей среде. Перед сдачей запустите скрипты по инструкции и вставьте собственные скриншоты в отчёт или приложите их отдельно.

Команды для запуска (после установки необходимых библиотек и генерации profile_pb2.py):

```
python step_json.py
python step_protobuf.py
python server.py
python client.py
```

Для TCP-теста сначала запускается server.py, затем в другом терминале client.py. В client.py по очереди меняется значение FMT на json, msgpack, cbor и protobuf.